This laboratory assignment accompanies the book, *Embedded Systems: Real-Time Interfacing to ARM Cortex M Microcontrollers, ISBN-13: 978-1463590154*, by Jonathan W. Valvano, copyright © 2026.

Goals

- To interface two DC motors using PWM so software can adjust delivered power
- To interface two tachometers using input capture so the software can measure speed
- To implement the digital controller processing with periodic interrupts
- To study the behavior of the system as parameters to the PI controller are adjusted
- To implement a Command Line Interpreter (CLI) that allows the operator to set controller parameters and to display the status of the motor when it is running.

Responsibilities:

- Student 1: Motors and interface hardware and PWM/direction outputs
- Student 2: Tach hardware and input capture measurement of two speeds
- Student 3: Digital Controller
- Student 4: User interface (CLI) via a UART to PC interface.
- ALL: Tinker with variations of the PI control loop

Team Size: 4

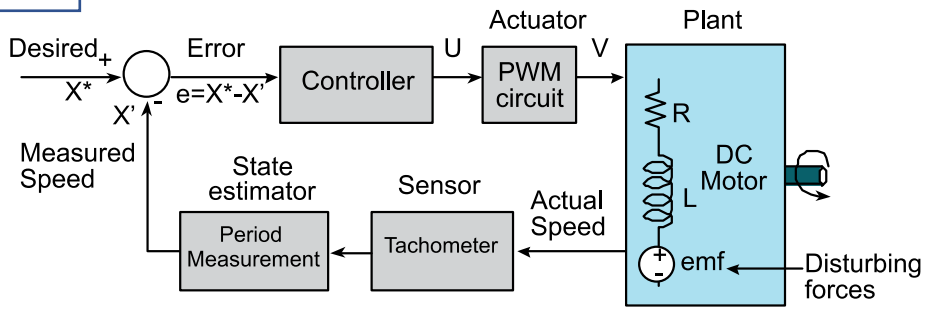


Figure 8.1: Lab 8 Block Diagram (there are two motors and two tachometers).

Review

- Power configurations for MSPM0G3507 LaunchPad in RSLK2 Description
- ebook Section 8.2 or textbook Section 6.1 about using input capture to measure period
- ebook Section 8.3 or textbook Section 6.5 about interfacing a DC motor with a DRV8847
- ebook Section 8.4 or textbook Section 6.3 about generating PWM outputs
- Pololu motor https://www.pololu.com/product/3675
- RSLK Description (especially look at how it is powered)
- ebook Section 8.7 or textbook Section 6.6 about proportional integral control
- PI Control notes from BYU proportional control

Starter projects

- TimerPWM
- InputCaptureInt
- Lab8
- TimerPeriodicInt
- RSLK

Required Hardware

LP-MSPM0G3507 $18.69
RSLK2 Robot,

Background

The advantages of DC motors include low cost, high speed, and high torque. However, they need sensors and closed-loop controllers to operate them with predictable responses. Applications of DC motors include electric cars, robotics, industrial machines (pumps, drills, mills, and lathes), consumer products (blender, washing machine, and AC/heaters) and medical devices (pumps). In this lab, you will spin two brushed DC motors (Figure 8.2) at a constant speed using a proportional-integral (PI) controller. The user interface will allow the operator to increase or decrease the desired speed. Two tachometers will be used to measure the motor speeds. A background periodic interrupt will execute the three steps of a digital controller.

1. Calculate error as the difference between actual and desired speed
2. Execute a control equation to determine the next output
3. Adjust the power to the actuator to drive the error to zero

The software will set the power delivered to the motor using pulse-width-modulation (PWM). The essence of an integral controller is the amount you add to the power is linearly related to the error. If the motor is spinning much too slowly, you will increase power a lot. If the motor is spinning just a little bit too slowly, you will increase power a little bit. If the motor is spinning much too quickly, you will decrease power a lot. If the motor is spinning just a little bit too quickly, you will decrease power a little bit. The input capture input and the PWM output must be written at a low level, like the book, without calling TivaWare driver code. No TivaWare code is allowed.

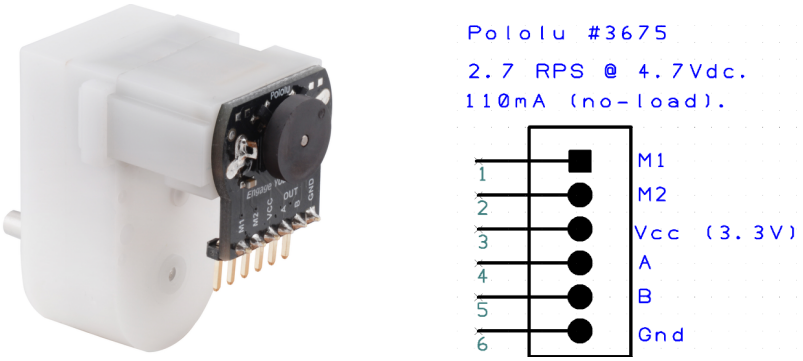


Figure 8.2. RSLK2 robot uses two Pololu item #3675. M1 and M2 connect to the DC motor and the other four wires are the tachometer. Tachometer signal B is not connected.

Figure 8.3 shows a simple data flow graph. Two tachometer sensors create square waves depending on the two motor speeds. Input capture interrupts measure left and right motor speeds. These data are fed to the motor controller, which adjusts the left and right duty cycles in attempt to spin both motors at the

same speed. The two PWM duty cycles are fed to the DRV8847 H-bridges to set the power to the two motors. If successful, the robot should move in a straight line.

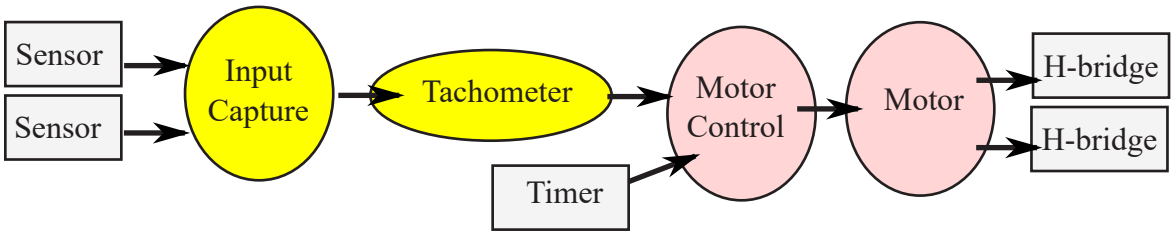


Figure 8.3. Simple data flow graph.

Figure 8.4 shows a more complex data flow graph. The operator on the PC can set the desired speed and controller parameters. If the robot triggers one of the four bump switches, the two motors should stop. System parameters can be observed on the PC and the LCD.

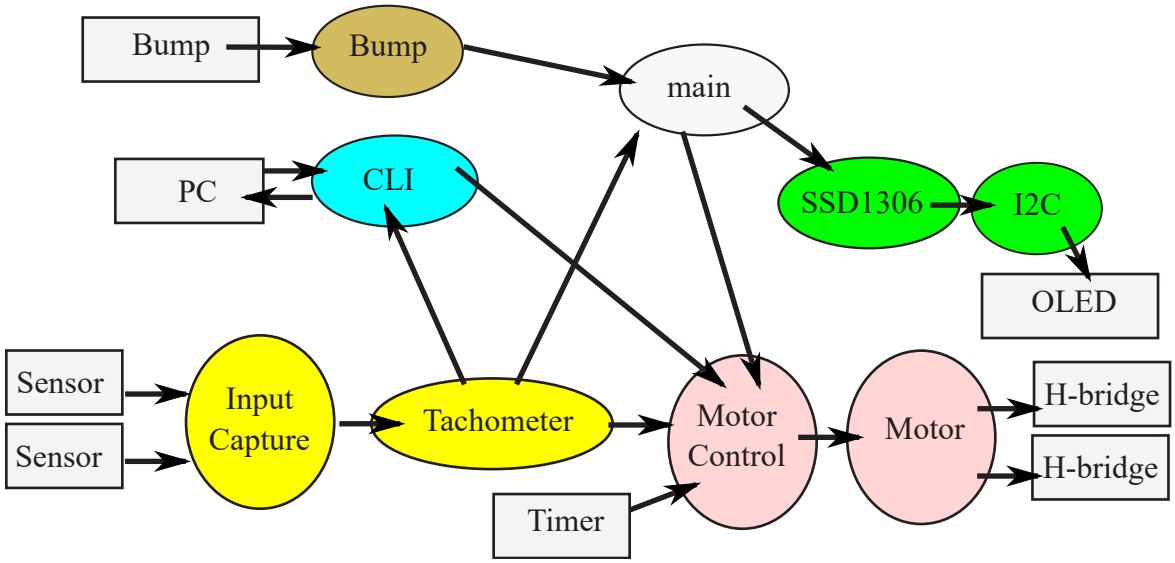


Figure 8.4. More detailed data flow graph.

Figure 8.5 shows a possible call graph. You are free to solve Lab 8 with whatever data flow graph or call graph as you which. Recall however, that modularity maximizes the number of modules and minimizes the coupling. **Bandwidth coupling** is defined as the amount of data flowing through the arrows of the data flow graph. **Invocation coupling** is defined as the number function calls occurring in the call graph over a fixed time interval.

Note: it is very bad style to connect modules using the **extern** modifier. Using **extern** creates a coupling that is hard to manage and impossible to quantify. If you find it impossible to solve a system without **extern**, then consider merging the two modules into one module.

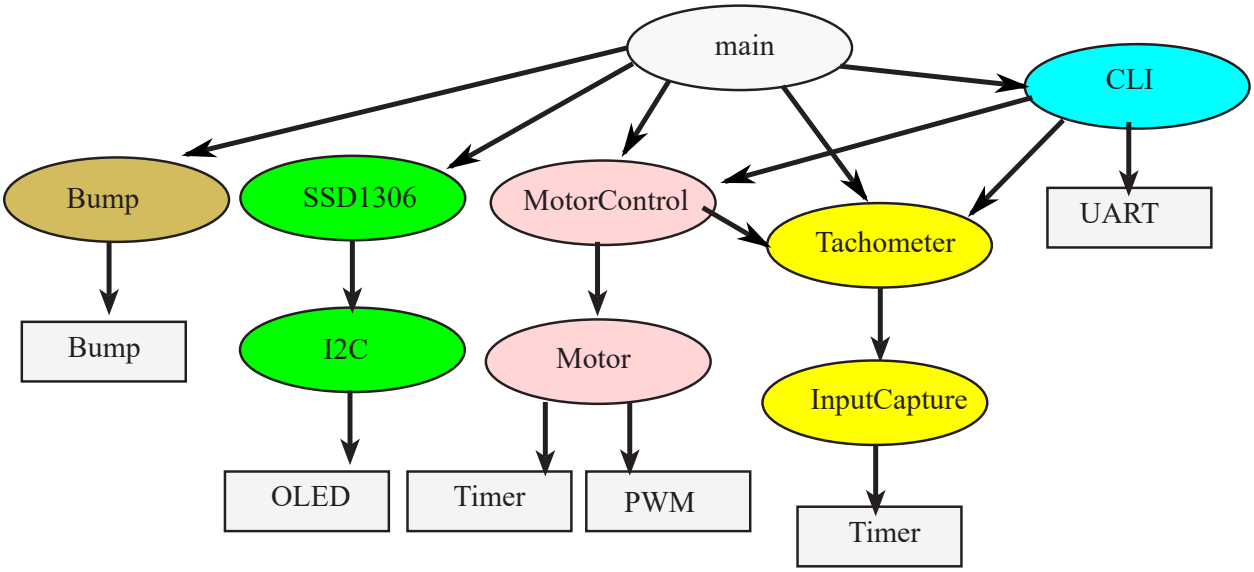


Figure 8.5. Possible Lab 8 call graph.

As a team edit the Timers tab in **MSPM0_Pins_Assignments.xlsx** file to make sure all modules can be combined into one system.

Student 1: Motor interface with PWM output

Program 8.1 shows the motor pins on the RSLK2 robot.

```
//   MSPM0
//   PB4  Motor_PWML, ML+, IN3, PWM M1PWM6
//   PB1  Motor_PWMR, MR+, IN1, PWM M1PWM7
//   PB0  Motor_DIR_L,ML-, IN4, GPIO 0 means forward, 1 means back
//   PB16 Motor_DIR_R,MR-, IN2, GPIO 0 means forward, 1 means back
```

Program 8.1. Motor pins on the RSLK2.

You must use an MSPM0 LaunchPad. The MOSFET transistors in the DRV8847 H-bridge will remove large voltage transients caused by the inductance of the motor. Voltage transients occur both when the power is applied and when it is removed. *Note*: V = L dI/dt depends on inductance L and the speed of the driver. Using Darlington transistor interface without snubber diodes, this peak can reach 100s of volts. Table 8.1 shows the operation of the DRV8847.

IN1	IN2	OUT1	OUT2	Function
0	0	Z	Z	Motor coast
0	1	L	H	Reverse
1	0	H	L	Forward
1	1	L	L	Brake

Table 8.1. DRV8847 operation.

To choose a PWM period you will need to know the time constant of your motor. Figure 8.6 is measured speed as a function of time as PWM is increased from 25 to 50%, with the wheels off the ground, using the program main7. We estimate the time constant of this motor to be about 75 ms. Therefore, if the PWM

period is 2 ms, the motor will only respond to the average power (duty cycle), and not to the individual high and low signals of the interface.

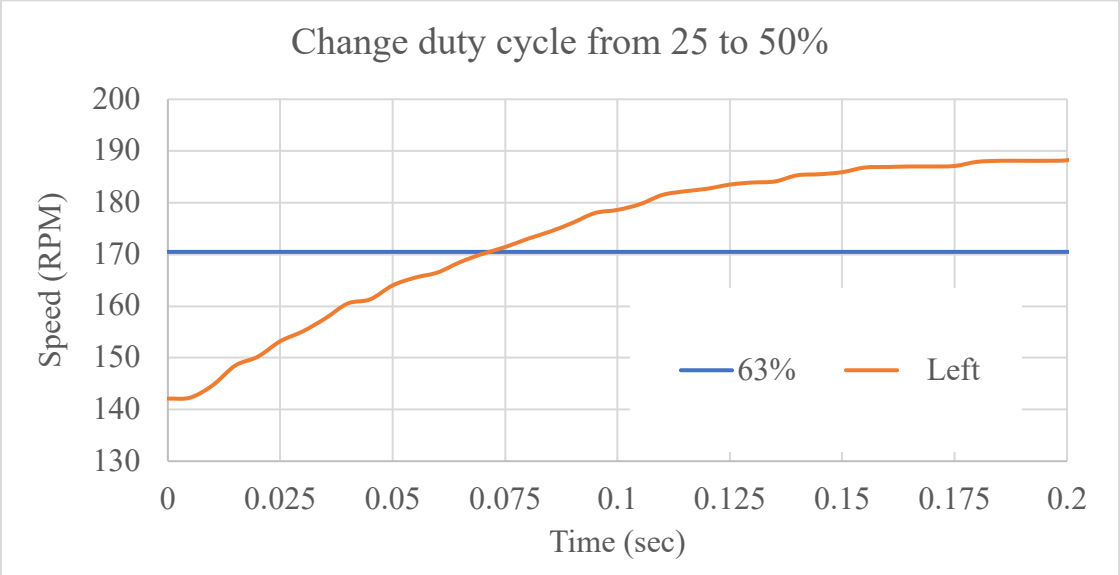


Figure 8.6. Step response of the DC motor used to estimate the time constant. This data was collected at 7.2V with wheels off the ground.

Student 2: Tach hardware and input capture measurement of speed

Program 8.2 shows the tachometer pins on the RSLK2 robot. The tachometer is interfaced to PB8 and PB12 using Timer A0. Remember to insert RSLK2 board jumpers J17 and J18 to connect the tach to the microcontroller.

```
//   MSPM0
//   PB8  ELA  TA0_C0
//   PB12 ERA  TA0_C1
```

Program 8.2. Tachometer pins on the RSLK2.

There is a 4.7k pullup to 3.3V for the tachometer signals ELA and ERA. Tachometer signals ELB and ERB are not connected. The motors have a gear box, so there are 360 tach pulses per rotation. Figure 8.7 shows a tachometer frequency of about 1kHz. This corresponds to a motor speed of

(1000 pulses/sec)*(1 rotation/360 pulses)*(60 sec / 1 min) = 166.7 RPM

Assume the input capture resolution is 1us. The raw input capture period measurement for the 1 kHz input will be 1000 clocks/pulse. Using a similar equation, we can calculate the motor speed as

(1000000 cycles/sec) *(60sec / 1min) *(1 rotation/360 pulses) /(1000 clocks/pulse) = 166.7 RPM

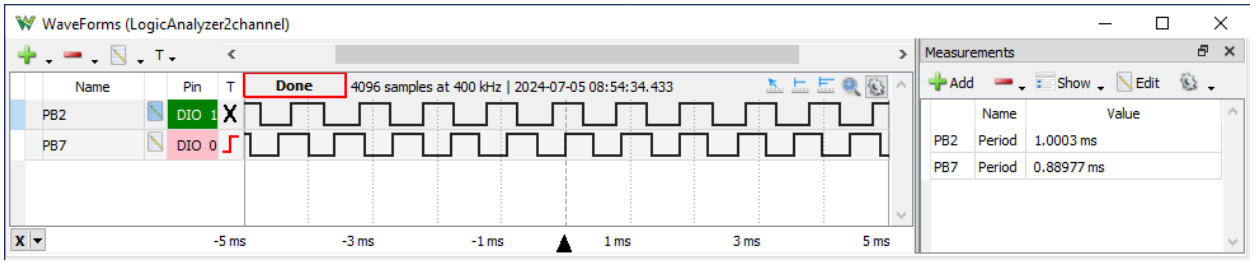


Figure 8.7. Tachometer output at full speed (MSPM0 will be pins PB8 and PB12).

The tachometer interface may require analog or digital filtering to improve SNR. However, be careful to minimize time delay in the filter. Time delays in the tachometer measurement may cause the controller to be unstable. For any digital filter, measure the step response. A simple measure of filter delay is to make input go from …x,x,x,x,y,y,y,y… and determine many samples it takes for the output of the filter to reach $x+(y-x)e^{-1}$. For example, the filter delay of a 5-wide median filter is 3 samples.

You will need a separate 100Hz periodic task to determine if the motor is spinning too slow to be accurate or not spinning at all.

Student 3: Digital Controller Design

A closed loop feedback is required because not all motors are created equal. Additionally, the friction faced by the robot can vary from robot to robot and from place to place. One of the first steps in designing a digital controller is to make an estimate of motor speed in RPM versus PWM duty cycle. This curve will help estimate the range of speeds that your controller will be able to manage. In other words, what speeds correspond to duty cycles from 25 to 75%? Note: this response is highly dependent on friction. We expect the usable range to be around 40 to 120 RPM, but your robot may be different. The design progression is

1. Incremental (stable but slow)
2. Integral (stable, a little faster)
3. Proportional-integral (may be unstable, fast)

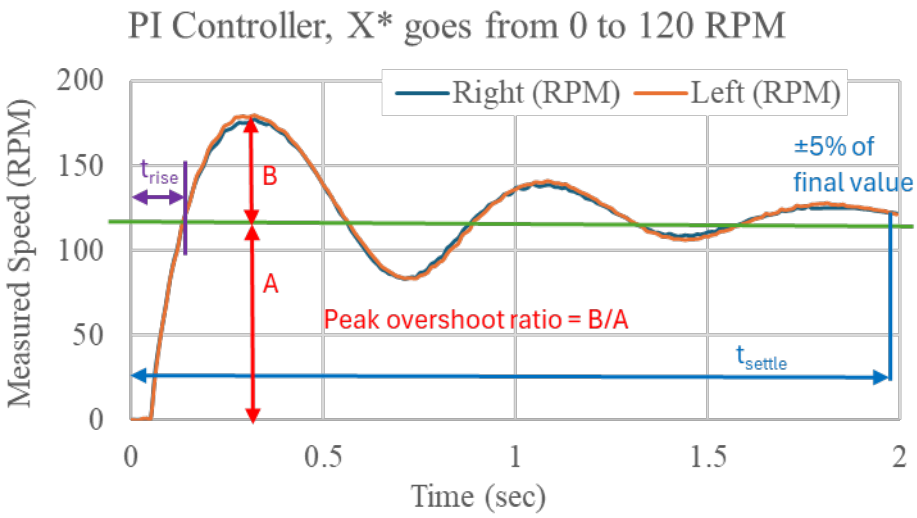


Figure 8.8. PI Controller Response showing underdamped behavior.

Student 4: UART <-> Command Line Interpreter (CLI) -> Motor Control

Create a mechanism to visualize and interact with the system using a Command Line Interpreter (CLI) connected to a UART port on the MSPM0. See the requirements document in the **Lab8Report** for details.

Preparation (do this before your lab period)

1. Edit the requirements document in the **Lab8Report** as needed. Assign team members to each task. The requirements document is meant to be fluid. Therefore, you should be editing it as you develop this robot controller.

2. Remove 5V jumper from your LaunchPad and make sure your LaunchPad has the 2-pin female jumper to connect to J7 on the robot board.

3. Review the RSLK design documents. Understand the microcontroller pin assignments. Notice how the robot is powered in debug mode and how it is powered in stand-alone mode.

4. Charge the 6 AA NiMH batteries and then run the RSLK test project to make sure the motors and sensors are operating properly. If it doesn't work, please see a TA for help rather than returning it to checkout for another, because all robots were tested prior to the start of the semester.

5. Design four software modules that implement the robot controller. More specifically, edit the four header files to reflect your design:
   - **Motor.h** PWM outputs to PB4 PB1, GPIO outputs to PB0 PB16
   - **Tachometer.h** Input capture inputs on PB8 PB12
   - **MotorControl.h** Review the linkage to the other modules
   - **CLI.h** UART I/O for operator control

All Students Procedure (do this during your lab period)

Perform all testing and measurements with the robot positioned so the wheels do not touch the ground.

Student 1 Procedure (do this during your lab period)

Deliverable 1

Develop and test software that creates the two PWM outputs with a fixed period anywhere in the range of 1 to 5ms. With a two-channel scope or logic analyzer, measure the outputs on PB4 and PB1, like Figure 8.9. When the controller is running the two PWMs will have different duty cycles, because the controller will adjust the PWMs so each motor spins at the same speed.

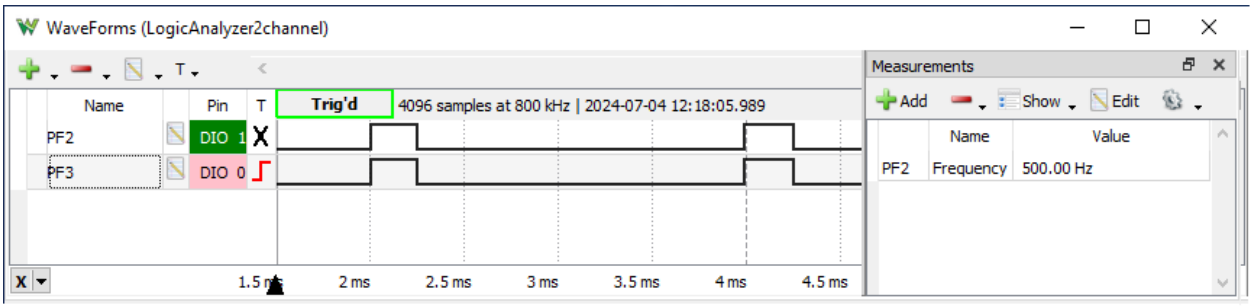


Figure 8.9. Two PWM outputs allow the system to adjust power to the motors (MSPM0 PWM pins to motor are PB4 and PB1).

Deliverable 2

When the motor is spinning forward, what two states from Table 8.1 are active? Using a dual trace scope measure ML+ and ML- like Figure 8.10. You should see three distinct regions of the traces (one where power is applied to the motor and two where no power is applied). Measure the voltage difference, V_m, when power is applied to the motor. Using two of the four Figures 7-4, 7-5, 7-6, and 7-7 from the DRV8847 data sheet, explain these three regions. Hint: scope trace 1 is ML+ and scope trace 2 is ML-.

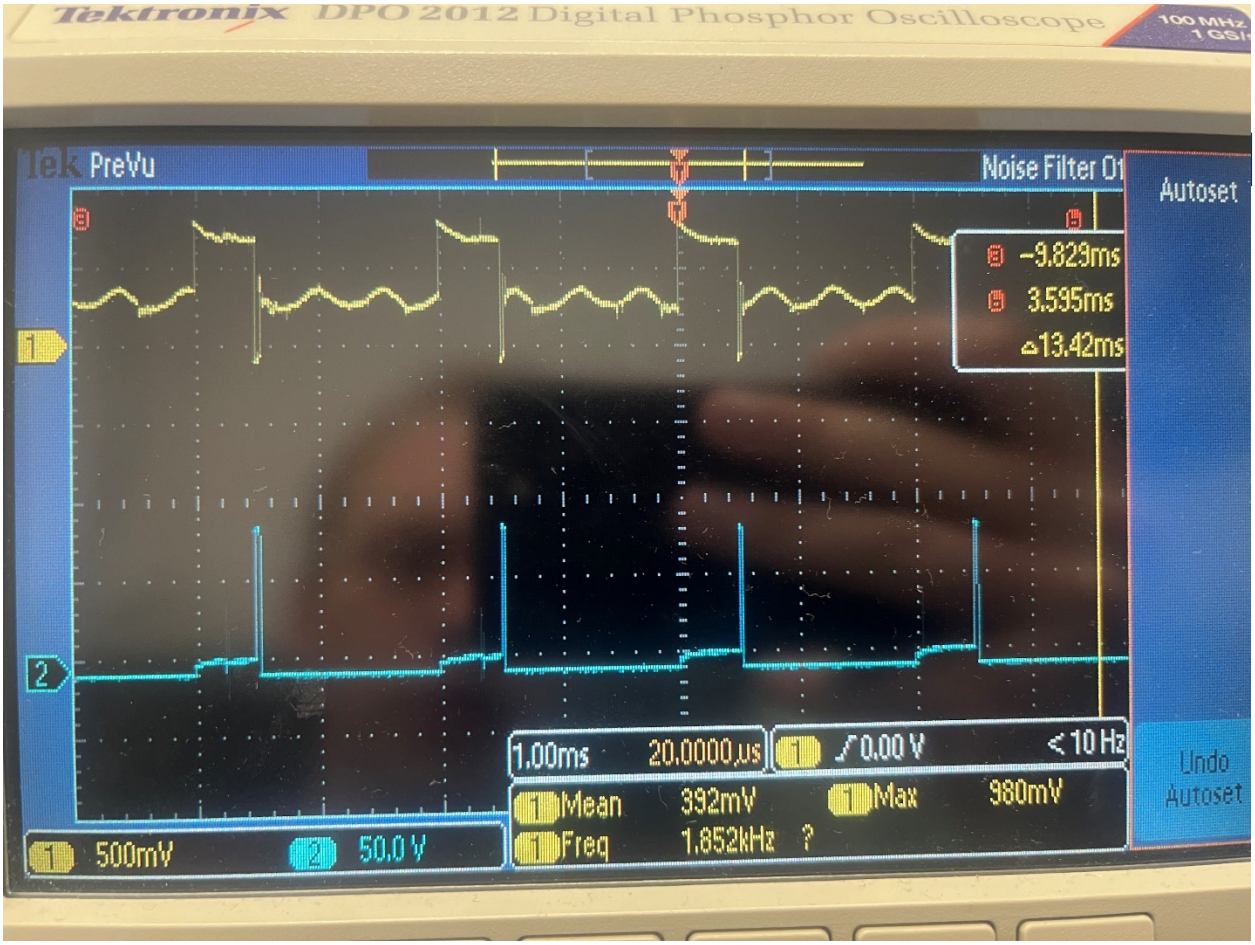


Figure 8.10. Voltage measured on the DC motor ML+ (yellow) and ML- (blue) with the DRV8847.

Deliverable 3

Using a DVM, measure the +7.2V current, I_{total}, (to entire robot) when both motors spin at 0%, 5%, 10%, 20%, 30%, 40%, and 50% duty cycle. There is a 2-pin jumper, J16, on the RSLK PCB to simplify the I_{total} measurement. Please contact your professor or TA if the duty=0 current is vastly different from 120mA. Subtracting off the current at 0%, calculate the current to each motor as

$$I_m(\text{duty}) = (I_{total}(\text{duty}) - I_{total}(0))/2$$

We estimate the average voltage to the motor from the duty cycle

$$V_{ave}(\text{duty}) = V_m * \text{duty, where duty} = \{0\%, 5\%, 10\%, 20\%, 30\%, 40\%, \text{and } 50\%\}$$

Create a graph like Figure 8.11 below. Does the data suggest the motor could be modeled as a simple resistive load?

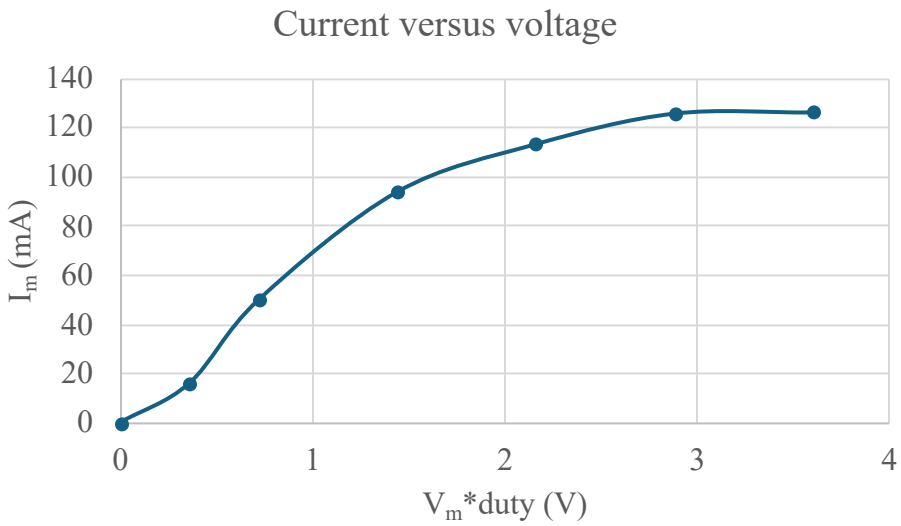


Figure 8.11: The current to one DC motor is plotted versus average voltage. Notice the DC motor does not behave like a resistor. This response will also be a function of the mechanical torque (or friction) applied to the shaft.

Interesting observation: For duty cycles from 20 to 50%, motor current is approximately equal. This means there is a linear relationship between duty cycle and motor power, $P_m = V_m*I_m*$duty.

Student 2 Procedure (do this during your lab period)

Develop and test the tachometer software that performs the left and right speed measurements. Speed measurement must occur in the background using an input capture interrupt and a periodic interrupt.

An extra pulse (glitch) will create two short input capture measurements and generate two large speed calculations. If you are getting extra pulses, observe the signal with a scope and add analog, digital, or software filtering. At a constant duty cycle, the speed measurements should be stable with good SNR.

Deliverable 4

Using a 2-channel scope, verify the tachometer generates digital signals like Figure 8.7.

Deliverable 5

Using the PWM output in open loop fashion measure the motor speed at duty = {0%, 5%, 10%, 20%, 30%, 40%, and 50%). Do this with the robot's wheels not touching the ground. Estimate the average motor power as $P_m = V_m*I_m*$duty. Create a graph like Figure 8.12.

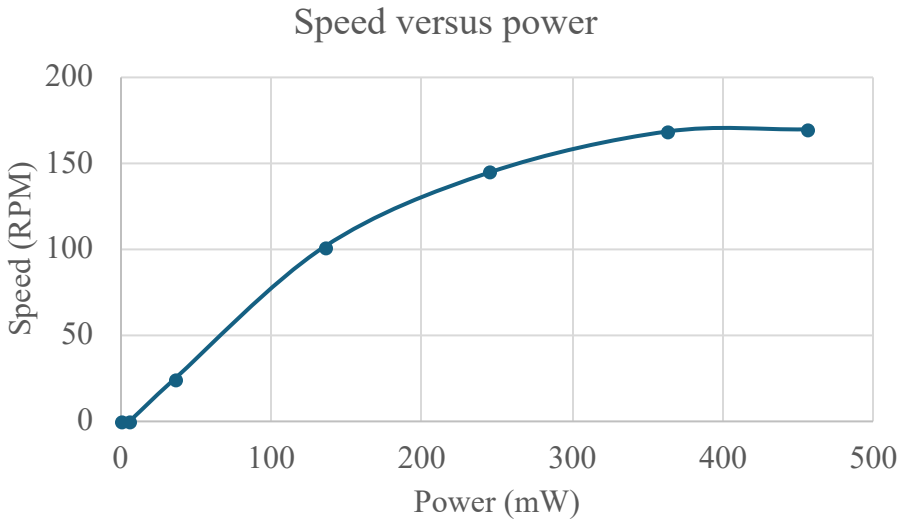


Figure 8.12: The motor speed is plotted versus applied power. Notice the three regions. From 0 to 5% duty cycle, the motor does not spin at all. From 10 to 40% duty cycles, the motor speed increases monotonically with power. For duty cycles above 40% the motor speed saturates. This response will also be a function of the mechanical torque (or friction) applied to the shaft.

Student 3 Procedure (do this during your lab period)

After you are sure the motor and tachometer interfaces are operating properly, test a simple incremental controller, see Figure 8.13. Choose a PWM period, a range of usable speeds, and a controller rate consistent with the data collected in the above two procedures. The range of the duty cycle U should match the capabilities of your software (100 to 9900us).

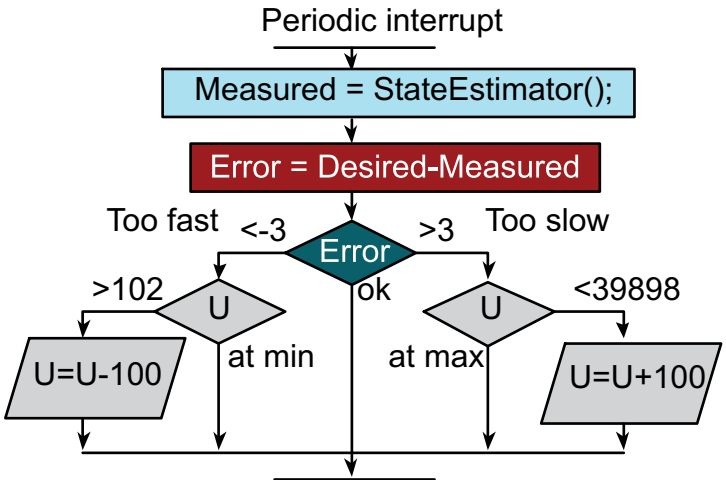


Figure 8.13: Incremental controller. Change any of these constants as you wish.

Debug the integral motor controller. The periodic interrupt-driven background thread will execute the digital controller, while the foreground thread initializes the system then performs human interface. An integral controller will be stable. If the speed oscillates wildly (all stop- full speed) then decrease the controller constant. If it is VERY slow to respond, increase the constant.

Debug the PI motor controller. Review this proportional control PowerPoint from BYU to understand controller design and performance metrics

- Rise time
- Setting time
- Peak overshoot Ratio

Student 4 Procedure (do this during your lab period)

Student 4 creates a mechanism to visualize and control the system using a UART connected to a PC using PUTTY (or similar terminal emulator). See requirements document for details. Dump into a buffer measured speed versus time immediately after the set point is changed and then display the buffer on the PC. These debugging dumps can be used to recreate graphs like Figures 8.14 to 8.17.

All Students Procedure (do this during your lab period)

We define **locking** or **stable** as the condition when the error eventually goes to approximately zero. Pick one change that is most interesting to the group (e.g., 0 to 120 RPM, 50 to 100 RPM, etc.)

Deliverable 6

Collect the speed versus time response of the system for three stable PI controller settings (like Figure 8.14). Label the three responses as

- over-damped,
- optimally-damped,
- critically-damped, or
- under-damped.

Calculate rise time, settling time, and peak overshoot ratio for the three stable settings.

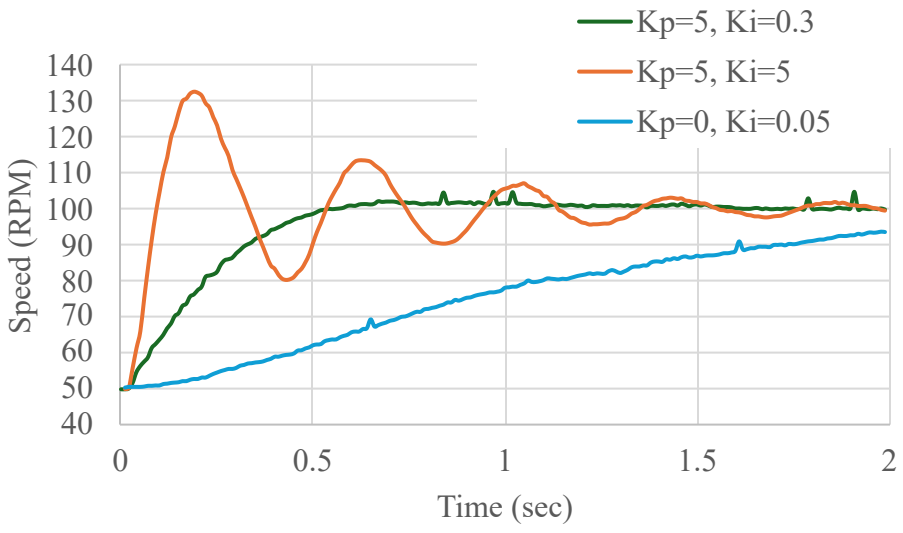


Figure 8.14: Measured speeds versus time as the set point is increased from 50 to 100 RPM.

The CLI user interface should facilitate changing controllers and controller parameters. You should be able to demonstrate without recompiling.

Student-all Procedure (do this during your lab period)

Deliverable 7

Measure the maximum time to execute each of your ISRs. Calculate CPU utilization for each task (motor control, tachometer, and main). Ignore the CLI and the UART communication to the PC

Deliverable 8

For your best controller, what is the steady state controller error (difference between desired and actual speed)?

Checkout (show this to the TA)

You should be able to demonstrate correct operation of the DC motor system. Be prepared to describe how your motor interface works. Demonstrate debugging features that allow you to visualize the software behavior. Show the TA the four modes specified in the Deliverables Section-4: Normal operation, Slow loop response time, Fast loop response time and Unstable mode.

All software and report files must be uploaded as instructed by your TA.

Hints

1. You should use a 100 Hz periodic interrupt to determine if the motor is stopped or spinning very slowly.
2. Please review the power distribution created by the jumper settings in the RSLK2 description.